score_experts.py — technical

`scripts/score_experts.py · 93 lines · scripts/score_experts.py`

Purpose

Ranking and selection from saved statistics (no model needed).

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`main`

```
def main() -> None
```

lines 24-89 · raises SystemExit · settings MODEL_PATH or HF_MODEL_ID, PRUNE_EXPERTS_PER_LAYER, PRUNE_LAYERS, PRUNE_UNOBSERVED_POLICY, PRUNING_METHOD

Script entry point: runs this stage end to end.

1. Require statistics and manifest; refuse if they belong to a different model source.
2. Rebuild `MoELayerInfo`s from the manifest; resolve `PRUNE_LAYERS`; `score_and_select` with `PRUNE_EXPERTS_PER_LAYER` and the unobserved policy.
3. Write `expert_scores.json` and `.csv`; write `expert_selection.json` with settings, the calibration fingerprint and per pruned layer: selected and unobserved experts, full ranking, layer description.
4. Log each layer's ranking as `expert:importance share` (selected marked with `*`).

Errors raised

Function	Exception	Message (f-string fields in braces)
`main`	SystemExit	Statistics were collected for {manifest['model_source']} but the configured model is {cfg.model_source}. Re-run collect_stats.py.